Development of a Web-Based Self-Driving Car Rental Support Platform Utilizing Java Servlet and SQL Server

First Author[1], Second Author[1], and Third Author[1]

FPT University, Da Nang, Vietnam
{auth1, auth2, auth3}@fpt.edu.vn

Abstract. The rapid expansion of the sharing economy has drastically transformed traditional personal vehicle utilization frameworks. This paper details the structural design and deployment of a comprehensive web-based self-driving car rental support platform built upon a Model-View-Controller (MVC) architecture leveraging Java Servlet, JSP, and SQL Server backend infrastructures. The application focuses heavily on automating vehicular contract flows, optimizing real-time fleet availability, and analyzing precise battery metrics during handover cycles. Furthermore, secure electronic transactions are simulated seamlessly via the VNPAY Sandbox framework to handle deposit configurations and automated user refunds. Experimental testing metrics suggest that the platform significantly optimizes transaction integrity and system efficiency across core operational stakeholders.

Keywords: Self-driving car rental · Java Servlet · VNPAY API · Fleet control · E-commerce architecture.

1 Introduction

Modern internet-driven ecosystems have successfully facilitated distributed peer-to-peer sharing dynamics, specifically within metropolitan transportation segments. Conventional car rental operations, however, remain bounded by fragmented information channels, which exposes vehicle fleets to logistical uncertainties regarding precise battery capacity reporting, availability indexing, and transparent security deposit handling.

To counter these core operational problems, this research introduces an enterprise-grade web platform utilizing Java Servlet, JSP, and relational SQL Server setups. The architectural deployment covers systemic security workflows, automated renting matrices, and automated e-commerce booking processing connected to a simulated VNPAY payment network.

The remainder of this paper is structured as follows: Section 2 models the core platform stakeholders. Section 3 outlines the functional use case mapping and details specific operational sub-systems. Finally, Section 4 concludes the paper and presents acknowledgments.

2 System Architecture and Stakeholder Modeling

Granular requirements engineering of the proposed ecosystem maps exactly 5 major system stakeholders (actors) who maintain direct interaction with the platform services:

1. **Guest:** Unauthenticated entities who access general catalogs of rental cars, check daily baseline price sheets, monitor charging networks, and view general policy guidelines.
2. **Renter:** Authenticated system users possessing full operational capability to book specific electric vehicles, process electronic deposit requirements, evaluate trip metrics, and submit star-rating feedback.
3. **Car Owner:** Fleet provisioning partners who register assets into active inventory pools, approve booking windows, and audit granular accounting and revenue charts.
4. **Administrator (Admin):** Global managers who control system configurations, handle listing approvals, monitor general relational databases, and authorize automated transaction refunds.
5. **Customer Services (CS):** Specialized staff handling identity validation protocols (KYC document checks), answering support tickets, and addressing user disputes without modifying root configurations.

3 Functional Use Case Mapping

The structural core of the platform is managed using 35 specific Use Cases divided into distinct logic layers as classified in Table 1.

Table 1. Platform Functional Block Segmentation Matrix.

Functional Block	**Core System Features**	**Targeted Primary Actors**
Operations Layer	Core Framework Parameters	All Registered Users
Authentication Block	User Management, KYC Verify	Guest, Renter, Vehicle Owner
Rental Booking	Vehicle Discovery, Deposit Processing	Renters, Vehicle Owners
Fleet Control	Battery Tracking, Contract Workflow	System Admins, Vehicle Owners
Support Controls	Discrepancy Auditing, Disputes	Customer Service Agents

3.1 Authentication and KYC Operations

Foundational access infrastructure manages user onboarding (UC01), secure session creation through email or Google OAuth (UC02), and strict identity verification protocols (UC04). Users must supply verified driver identification and national identity cards to protect platform property during the Know Your Customer (KYC) cycle.

3.2 Rental Booking and Payment Integration

The vehicle ordering workflow automates peer-to-peer reservations. Renters browse available electric fleets, select individual booking dates (UC13), and initiate secure deposit workflows (UC14). To enforce financial integrity, the platform integrates the VNPAY Sandbox payment gateway API. The baseline transaction calculation for an active booking session is governed by the following system billing equation:

$$T_{total} = (P_{day} \times N_{days}) + D_{deposit} + F_{service} \tag{1}$$

where T_{total} represents the total transaction cost, P_{day} is the daily baseline price, N_{days} denotes the rental duration, $D_{deposit}$ represents the secure electronic safety deposit, and $F_{service}$ is the platform maintenance fee. Upon order verification, the VNPAY core processor logs the simulated digital currency transfer, which guarantees immediate funds isolation until vehicle recovery.

3.3 Fleet Control and Vehicular Lifecycle Management

Asset monitoring requires rigid real-time data persistence. Vehicle owners publish vehicle technical data sheets including maximum range, registration documents, and rental windows (UC21). System administrators retain global auditing privileges to approve or reject new listings (UC27). Crucially, the platform enforces data collection during vehicle delivery and return cycles (UC18). The actual battery state-of-charge (SoC) degradation curve is indexed to map vehicular battery integrity during handover periods, protecting owner assets from unauthorized high-discharge misuse.

3.4 Customer Support and Dispute Resolution

The support layer establishes an unbacked administrative boundary for specialized staff members. Customer Service agents access distinct communication pipes to audit user identities (UC31) and process tickets (UC32). In cases where real-time fleet discrepancies arise—such as vehicle damage reports or delayed returns—CS agents execute dispute resolution protocols (UC34). They moderate user feedback and flag high-risk transactional logs for subsequent Administrator review without interacting with root database engines.

3.5 System Control and Operational Parameters

Please avoid using low-resolution rasterized diagrams for architecture modeling. Use vector layouts instead to visualize the system framework (see Fig. 1).

Theorem 1. *Automated Contract State Theorem: A booking state can change to 'active' if and only if the identity verification status equals 'KYC Approved' and the VNPAY core processor verifies successful deposit validation.*

Proof. Verifying structural logs guarantees that unverified guest entities cannot alter data partitions or view restricted owner financial summaries.

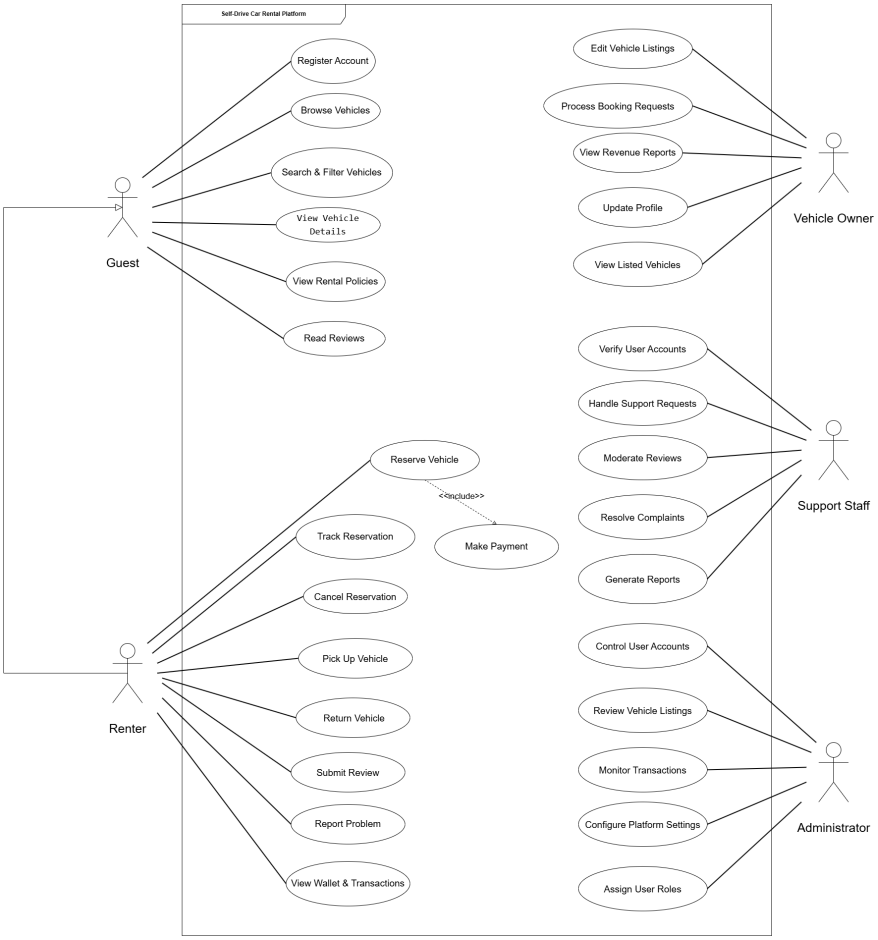

Fig. 1. System Use Case flow rendering data transactions for target stakeholders.

Acknowledgments. The project development team acknowledges the technical infrastructure and guidance provided by FPT University, Da Nang, Vietnam.

Disclosure of Interests. The authors state that there are no competing financial or personal interests that could influence the work detailed in this manuscript.

References

1. VNPAY: VNPAY Sandbox Payment Gateway Integration Documentation (2026), https://sandbox.vnpayment.vn/apis/, last accessed 2026/06/08
2. Schildt, H.: Java: The Complete Reference. 12th edn. McGraw Hill, New York (2023)
3. Microsoft: SQL Server Relational Database Management Guidelines for Enterprise Applications (2024)
4. Le, T., Nguyen, M.: E-Commerce Software Architecture Optimization in Urban Transport Networks. Journal of Software Engineering **5**(2), 45–58 (2025)